Hosting & server management

A small managed deployment first, with explicit responsibilities and cost controls.

PROPOSED • Not implemented • Scientific goal, scoring weights and hosting provider remain undecided

01

Proposed deployment topology

Static frontend on a CDN; API in a managed Linux container; managed PostgreSQL/PostGIS; object storage for assets. Run imports as separate batch jobs. Add a dedicated worker and durable queue only when long-running route workloads justify them.

02

Runtime management

Pin dependencies and build immutable images. Configure process startup, health checks, restart policy and graceful shutdown. Size API concurrency around memory and database connections. Offload CPU-heavy terrain work so it cannot block catalog requests.

03

Environment and networking

Separate local development, staging and production configuration. Keep database private with least-privilege credentials. Use managed TLS termination and explicit allowed origins. Record provider, region, domain, monthly budget and ownership as team decisions.

04

Monitoring and incident handling

Capture structured logs with request/job IDs and dataset/model versions. Track API errors, latency, job failures, queue depth, storage and database health. Alert on persistent failures. Keep a short runbook: triage, pause jobs, rollback and restore.

05

Backups and recovery

Schedule database backups and define retention plus recovery targets with the team. Test restoring into an isolated environment. Version scientific assets; keep rebuild scripts and source checksums. Persist jobs and state outside ephemeral container filesystems.

06

Capacity and cost management

Cap query area, candidate count, raster resolution, job runtime and concurrent jobs. Use cache keys containing mission and dataset versions. Watch database, storage, bandwidth and imagery-provider quotas. Keep a small precomputed demo dataset for upstream outages.

Read technical-brief.md for linked sources, assumptions and competition timing.